

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

Problem Statement

Develop a Reinforcement Learning framework that demonstrates the interaction between an agent and its environment using states, actions, rewards, and cumulative rewards.

Problem Solving

The agent interacts with the environment by selecting actions based on the current state. After each action, it receives a reward and moves to the next state. The cumulative reward is calculated to evaluate the quality of the decisions.

Program

```
import random
```

```
# States
states = ["Start", "Middle", "Goal"]

# Actions
actions = ["Move Left", "Move Right"]

# Rewards
reward = {
    "Start": 0,
    "Middle": 10,
    "Goal": 50
}

state = "Start"
total_reward = 0

print("Agent-Environment Interaction")

for i in range(3):
    action = random.choice(actions)
    print("Current State:", state)
    print("Action:", action)

    if state == "Start":
        state = "Middle"
    elif state == "Middle":
        state = "Goal"

    total_reward += reward[state]

    print("Next State:", state)
    print("Reward:", reward[state])
    print()

print("Cumulative Reward:", total_reward)
```

```
[1] 0s  "Goal": 50
}

state = "Start"
total_reward = 0

print("Agent-Environment Interaction")

for i in range(3):
    action = random.choice(actions)
    print("Current State:", state)
    print("Action:", action)

    if state == "Start":
        state = "Middle"
    elif state == "Middle":
        state = "Goal"

    total_reward += reward[state]

    print("Next State:", state)
    print("Reward:", reward[state])
    print()

print("Cumulative Reward:", total_reward)
```

*** Agent-Environment Interaction
Current State: Start
Action: Move Right
Next State: Middle
Reward: 10

Current State: Middle
Action: Move Right
Next State: Goal
Reward: 50

Current State: Goal
Action: Move Right
Next State: Goal
Reward: 50

Cumulative Reward: 110

Conclusion

The RL framework successfully demonstrated agent–environment interaction, showing how cumulative rewards guide the agent toward better decision-making.

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

Problem Statement

Model a decision-making problem as a Markov Decision Process (MDP) using states, actions, transition probabilities, reward function, and discount factor.

Problem Solving

An MDP model was created with defined states and actions. Transition probabilities determined state changes, rewards evaluated actions, and the discount factor considered future rewards.

Program

```
import numpy as np
import random
```

```

# States
states = ["Home", "Office", "Destination"]

# Actions
actions = ["Move", "Stay"]

# Transition Probability Matrix
transition = np.array([
    [0.2, 0.8],
    [0.3, 0.7],
    [1.0, 0.0]
])

# Rewards
rewards = [0, 10, 50]

# Discount Factor
gamma = 0.9

state = 0
total_reward = 0

print("Markov Decision Process (MDP)\n")

for i in range(3):

    action = random.randint(0, 1)

    print("Current State:", states[state])
    print("Action:", actions[action])

    total_reward += rewards[state]

    if state < 2:
        state += 1

    print("Reward:", rewards[state])
    print()

print("Discount Factor ( $\gamma$ ):", gamma)
print("Total Reward:", total_reward)

```

```
[2]
✓ Os
print("Markov Decision Process (MDP)\n")

for i in range(3):

    action = random.randint(0, 1)

    print("Current State:", states[state])
    print("Action:", actions[action])

    total_reward += rewards[state]

    if state < 2:
        state += 1

    print("Reward:", rewards[state])
    print()

print("Discount Factor (γ):", gamma)
print("Total Reward:", total_reward)
```

▼ ... Markov Decision Process (MDP)

Current State: Home
Action: Stay
Reward: 10

Current State: Office
Action: Stay
Reward: 50

Current State: Destination
Action: Move
Reward: 50

Discount Factor (γ): 0.9
Total Reward: 60

Conclusion

The MDP model successfully represented sequential decision-making and showed how future rewards influence optimal decisions.

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

Problem Statement

Implement policy and value functions using the Bellman Equation to evaluate long-term rewards.

Problem Solving

The Bellman Equation was applied to compute state values. The policy selected actions that maximized expected future rewards.

Program

Q3 - Policy and Value Functions using Bellman Equation

```

import random

# States
states = ["S1", "S2", "Goal"]

# Rewards
rewards = [10, 20, 50]

# Discount Factor
gamma = 0.9

# Initial State Values
V = [0, 0, 0]

print("Bellman Value Updates\n")

# Bellman Equation Update
for i in range(2, -1, -1):
    if i == 2:
        V[i] = rewards[i]
    else:
        V[i] = rewards[i] + gamma * V[i + 1]

# Display State Values
for i in range(3):
    print("State:", states[i])
    print("State Value V(s):", round(V[i], 2))
    print()

# Policy
print("Optimal Policy")
print("S1 -> Move to S2")
print("S2 -> Move to Goal")
print("Goal -> Stop")

```

```
[3] | # Discount Factor
✓ Os | gamma = 0.9

# Initial State Values
V = [0, 0, 0]

print("Bellman Value Updates\n")

# Bellman Equation Update
for i in range(2, -1, -1):
    if i == 2:
        V[i] = rewards[i]
    else:
        V[i] = rewards[i] + gamma * V[i + 1]

# Display State Values
for i in range(3):
    print("State:", states[i])
    print("State Value V(s):", round(V[i], 2))
    print()

# Policy
print("Optimal Policy")
print("S1 -> Move to S2")
print("S2 -> Move to Goal")
print("Goal -> Stop")

... Bellman Value Updates

State: S1
State Value V(s): 68.5

State: S2
State Value V(s): 65.0

State: Goal
State Value V(s): 50

Optimal Policy
S1 -> Move to S2
S2 -> Move to Goal
Goal -> Stop
```

Conclusion

The policy and value functions effectively identified optimal actions by estimating long-term returns.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

Problem Statement

Compare ϵ -greedy and softmax exploration strategies to balance exploration and exploitation.

Problem Solving

The ϵ -greedy method randomly explored actions with a small probability, while the softmax method selected actions based on probability distributions derived from Q-values.

Program

```
import numpy as np
import random
```

```
# Actions
```

```

actions = ["Action A", "Action B", "Action C"]

# Estimated Q-values
Q = np.array([10, 20, 15])

# Epsilon value
epsilon = 0.2

print("Q-values:", Q)

#  $\epsilon$ -Greedy Strategy
if random.random() < epsilon:
    action = random.randint(0, 2)
    print("\n $\epsilon$ -Greedy: Exploring")
else:
    action = np.argmax(Q)
    print("\n $\epsilon$ -Greedy: Exploiting")

print("Selected Action:", actions[action])

# Softmax Strategy
temperature = 1.0
probabilities = np.exp(Q / temperature) / np.sum(np.exp(Q / temperature))

print("\nSoftmax Probabilities:")
for i in range(len(actions)):
    print(actions[i], ":", round(probabilities[i], 3))

softmax_action = np.argmax(probabilities)

print("\nSoftmax Selected Action:", actions[softmax_action])

```

```
| 1s
# Epsilon
epsilon = 0.2

# ε-Greedy Selection
if random.random() < epsilon:
    action = random.randint(0, 2)
    print("ε-Greedy: Exploring")
else:
    action = np.argmax(Q)
    print("ε-Greedy: Exploiting")

print("Selected Action:", actions[action])

# Softmax Selection
temperature = 1

exp_q = np.exp(np.array(Q) / temperature)
prob = exp_q / np.sum(exp_q)

print("\nSoftmax Probabilities:")
for i in range(len(actions)):
    print(actions[i], ":", round(prob[i], 3))

best = np.argmax(prob)
print("\nSoftmax Selected Action:", actions[best])

*** ε-Greedy: Exploiting
Selected Action: Right

Softmax Probabilities:
Left : 0.0
Right : 0.999
Forward : 0.001

Softmax Selected Action: Right
```

Conclusion

Both exploration strategies successfully balanced exploration and exploitation, improving the learning performance of the agent.

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

Problem Statement

Demonstrate how reward shaping accelerates learning and prevents suboptimal policies.

Problem Solving

Intermediate rewards were added to guide the agent toward the goal, encouraging better decisions during training.

Program

```
import random

# States
states = ["Start", "Middle", "Goal"]

# Rewards with Reward Shaping
rewards = {
    "Start": 0,
    "Middle": 20,
    "Goal": 100
}

state = "Start"
total_reward = 0

print("Reward Shaping in Reinforcement Learning\n")

for step in range(3):

    print("Current State:", state)

    if state == "Start":
        state = "Middle"
    elif state == "Middle":
        state = "Goal"

    reward = rewards[state]
    total_reward += reward

    print("Next State:", state)
    print("Reward:", reward)
    print()

print("Total Reward:", total_reward)

if total_reward >= 120:
    print("Optimal Policy Learned")
else:
    print("Suboptimal Policy")
```

```
[6] print("Reward Shaping in Reinforcement Learning\n")
✓ Os
for step in range(3):

    print("Current State:", state)

    if state == "Start":
        state = "Middle"
    elif state == "Middle":
        state = "Goal"

    reward = rewards[state]
    total_reward += reward

    print("Next State:", state)
    print("Reward:", reward)
    print()

    print("Total Reward:", total_reward)

    if total_reward >= 120:
        print("Optimal Policy Learned")
    else:
        print("Suboptimal Policy")
```

```
*** Reward Shaping in Reinforcement Learning

Current State: Start
Next State: Middle
Reward: 20

Current State: Middle
Next State: Goal
Reward: 100

Current State: Goal
Next State: Goal
Reward: 100

Total Reward: 220
Optimal Policy Learned
```

Conclusion

Reward shaping improved learning speed and helped the agent avoid suboptimal policies.

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

(5 marks)

Problem Statement

Implement Reinforcement Learning for robotics while considering safety, sample efficiency, and environmental variability.

Problem Solving

The robot learned safe navigation by selecting actions based on rewards while adapting to different environmental conditions.

Program

```

import numpy as np

import random

# Robot States
states = ["Safe", "Obstacle", "Goal"]

# Actions
actions = ["Move Forward", "Turn Left", "Turn Right"]

# Reward Matrix
rewards = np.array([
    [20, 10, 10],    # Safe
    [-20, 15, 15],   # Obstacle
    [50, 50, 50]     # Goal
])

# Initialize Q-Table
Q = np.zeros((3, 3))

# Parameters
alpha = 0.8
gamma = 0.9
epsilon = 0.2

# Training
for episode in range(100):

    state = random.randint(0, 2)

    #  $\epsilon$ -Greedy Action Selection
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = np.argmax(Q[state])

    next_state = random.randint(0, 2)

    # Q-Learning Update
    Q[state][action] += alpha * (
        rewards[state][action] +
        gamma * np.max(Q[next_state]) -
        Q[state][action]
    )

print("Learned Q-Table:\n")
print(Q)

print("\nBest Robot Action:")
for i in range(len(states)):

```

```
print(states[i], "->", actions[np.argmax(Q[i])])
```

```
epsilon = 0.1

# Training
for episode in range(100):

    state = random.randint(0, 2)

    # ε-Greedy Action Selection
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = np.argmax(Q[state])

    next_state = random.randint(0, 2)

    # Q-Learning Update
    Q[state][action] += alpha * (
        rewards[state][action] +
        gamma * np.max(Q[next_state]) -
        Q[state][action]
    )

print("Learned Q-Table:\n")
print(Q)

print("\nBest Robot Action:")
for i in range(len(states)):
    print(states[i], "->", actions[np.argmax(Q[i])])
```

Learned Q-Table:

```
[[264.97623923 245.21450215 96.99026541]
 [105.84260776 188.59732147 272.53508856]
 [253.81329914 66.02251878 295.29964225]]
```

Best Robot Action:

```
Safe -> Move Forward
Obstacle -> Turn Right
Goal -> Turn Right
```

Conclusion

The RL agent successfully improved robotic decision-making while addressing safety and environmental challenges.

Code of Conduct:

I _____ M.Samanvi (192425318) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Samanvi

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately